IUT d'Aix-Marseille Université - site d'Arles
Département Informatique
2ème année

R3.04 - Qualité de développement

TP n°5

Gestionnaire de contacts avec JavaFX
Explication de chaque question

Ce document reprend les 16 questions du TP dans l'ordre. Pour chaque question on donne le code écrit et on explique pourquoi on l'écrit comme ça. On n'utilise que ce qui est dans le cours (chapitres 1 à 14) ou ce que le sujet du TP demande d'utiliser. La référence au cours est donnée à chaque fois.

Le projet complet se trouve dans le dossier **TP05/GestionContacts** du dépôt. Il contient 3 classes dans le package **com.mycompany.gestioncontacts** :

- **App** : la fenêtre principale et toutes les actions (fichier **App.java**)
- **Contact** : un contact avec ses 4 informations (fichier **Contact.java**)
- **SystemInfo** : créée par NetBeans, on n'y touche pas

Question 1 : créer le projet

Dans NetBeans : **File** puis **New Project**, catégorie **Java with Maven**, et on choisit **Simple JavaFX Maven Archetype**. On l'appelle **GestionContacts**.

Pourquoi "Simple" et pas "FXML" ? Avec FXML, l'interface est décrite dans un fichier XML (fait avec Scene Builder). Avec "Simple", l'interface est écrite directement en Java, ce que demande le TP.

Cours : chapitre 11, diapos 14 et 15.

Question 2 : le code créé par NetBeans

NetBeans crée la classe **App** qui hérite de **Application**. Il faut bien comprendre les 3 notions de base de JavaFX :

- **Stage** : une fenêtre. La fenêtre principale est reçue en paramètre de **start()**.
- **Scene** : le contenu de la fenêtre (l'arbre des composants).
- **Node** : chaque composant (bouton, label, layout...).

```
public class App extends Application {

    @Override
    public void start(Stage stage) {      // appelée automatiquement au démarrage
        ...
        stage.setScene(scene);             // on met la scène dans la fenêtre
        stage.show();                      // on affiche la fenêtre
    }

    public static void main(String[] args) {
        launch();                          // lance JavaFX puis start()
    }
}
```

Remarque : NetBeans écrit **var scene = new Scene(...)**. Le mot **var** laisse Java deviner le type tout seul, ici c'est exactement pareil que **Scene scene = new Scene(...)**.

Cours : chapitre 11, diapos 12, 16, 17 et 18.

Question 3 : le titre de la fenêtre

```
stage.setTitle("Gestion de contacts");
```

setTitle() est une méthode de **Stage** : elle met le texte dans la barre de titre de la fenêtre.

Cours : chapitre 11, diapo 18 (primaryStage.setTitle).

Question 4 : le BorderPane

On enlève le label et le StackPane créés par NetBeans, et on met un **BorderPane** à la place :

```
import javafx.scene.layout.BorderPane;

BorderPane borderPane = new BorderPane();
...
var scene = new Scene(borderPane, 640, 480);
```

Un BorderPane découpe la fenêtre en 5 zones : haut, bas, gauche, droite et centre. C'est pratique ici car on va mettre la table au centre (question 8) et les boutons en bas (question 9). 640 et 480 sont la largeur et la hauteur de la fenêtre en pixels.

Cours : chapitre 11, diapo 39.

Question 5 : créer la classe Contact

Clic droit sur le projet, **New** puis **Java Class**, nom **Contact**. Il faut mettre le même package que **App.java** : **com.mycompany.gestioncontacts**.

Pourquoi le même package ? Les deux classes doivent se connaître. Si elles sont dans le même package, **App** peut utiliser **Contact** sans faire d'import. En plus, le projet utilise un fichier **module-info.java** qui déclare ce package : une classe mise ailleurs ne serait pas vue par JavaFX.

Cours : chapitre 8 (les paquetages).

Question 6 : les données membres de Contact

On veut 4 données membres : **lastName** (nom), **firstName** (prénom), **email** et **phone** (téléphone). Au lieu de **String**, on prend **StringProperty** :

```
import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;

private StringProperty lastName = new SimpleStringProperty("");
private StringProperty firstName = new SimpleStringProperty("");
private StringProperty email = new SimpleStringProperty("");
private StringProperty phone = new SimpleStringProperty("");
```

Pourquoi ? Une **propriété** JavaFX est une valeur **observable** : quand elle change, ceux qui la surveillent sont prévenus. Ici, celui qui surveille c'est la **TableView**. Donc quand on modifiera un contact (question 11), la table se mettra à jour toute seule, sans écrire de code en plus.

On met **private** pour respecter l'encapsulation : on ne touche aux données que par les méthodes.

Le constructeur reçoit des **String** normales et les range dans les propriétés avec **set()** :

```
public Contact(String ln, String fn, String em, String ph) {
    lastName.set(ln);
    firstName.set(fn);
    email.set(em);
    phone.set(ph);
}
```

Les méthodes d'accès : pour chaque donnée membre on écrit 3 méthodes. Exemple pour lastName :

```
public String getLastName() { return lastName.get(); }      // lire la valeur
public void setLastName(String v) { lastName.set(v); }      // changer la valeur
public StringProperty lastNameProperty() { return lastName; } // la propriété
```

Attention au nom de la 3ème méthode : c'est **le nom de la variable + Property**, donc **lastNameProperty**, **firstNameProperty**, **emailProperty** et **phoneProperty**. C'est obligatoire : à la question 8, **PropertyValueFactory<>("lastName")** va chercher tout seul (par réflexion) une méthode qui s'appelle exactement **lastNameProperty**. Si le nom est faux, la colonne reste vide, sans message d'erreur.

Cours : chapitre 11, diapos 54, 59 et 60 (classe Person avec des propriétés) ; chapitre 3 pour private, le constructeur et les accesseurs.

Question 7 : la liste observable

Dans la classe **App**, en donnée membre (en dehors de **start()**), on déclare :

```
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;

private ObservableList<Contact> contacts = FXCollections.observableArrayList();
```

Une **ObservableList** s'utilise comme une **ArrayList** (add, remove, get, size...) mais elle **prévient** les composants JavaFX quand son contenu change. On la donnera à la TableView : si on ajoute un contact dans la liste, il apparaît dans la table ; si on en enlève un, il disparaît. La liste est la référence, la table ne fait que l'afficher.

On la met en donnée membre car elle sert dans plusieurs méthodes : **start()**, **openEditor()**, l'export et l'import.

Remarque : c'est exactement le design pattern **Observer** du chapitre 14 : la liste est le sujet, la table est l'observateur, et elle est mise à jour automatiquement quand le sujet change.

Cours : chapitre 11, diapos 28 et 29 ; chapitre 14, diapo 102 (Observer).

Question 8 : la TableView

On crée la table et ses 4 colonnes. Une colonne a deux types entre < > : le type d'une ligne (**Contact**) et le type d'une case (**String**).

```
TableView<Contact> table = new TableView<>();

TableColumn<Contact, String> lastNameCol = new TableColumn<>("Nom");
lastNameCol.setCellValueFactory(new PropertyValueFactory<>("lastName"));

TableColumn<Contact, String> firstNameCol = new TableColumn<>("Prénom");
firstNameCol.setCellValueFactory(new PropertyValueFactory<>("firstName"));

TableColumn<Contact, String> mailCol = new TableColumn<>("Email");
mailCol.setCellValueFactory(new PropertyValueFactory<>("email"));

TableColumn<Contact, String> phoneCol = new TableColumn<>("Téléphone");
phoneCol.setCellValueFactory(new PropertyValueFactory<>("phone"));

table.getColumns().addAll(lastNameCol, firstNameCol, mailCol, phoneCol);
table.setItems(contacts);

borderPane.setCenter(table);
```

- **new TableColumn<>("Nom")** : le texte entre guillemets est le titre de la colonne.
- **setCellValueFactory(...)** : dit à la colonne quelle donnée de Contact elle affiche. **"lastName"** veut dire "appelle lastNameProperty()" (voir question 6).
- **getColumns().addAll(...)** : ajoute les colonnes, dans l'ordre où on les donne. J'ai mis l'ordre de l'énoncé : Nom, Prénom, Email, Téléphone (le même que le fichier CSV).
- **setItems(contacts)** : relie la table à la liste observable. C'est cette ligne qui fait que la table se met à jour toute seule.
- **borderPane.setCenter(table)** : la table prend la zone du centre, donc tout l'espace libre.

Cours : chapitre 11, diapos 30, 31 et 32 ; diapo 39 pour setCenter.

Question 9 : la barre de boutons

```
Button buttonAdd = new Button("Ajouter");
Button buttonModify = new Button("Modifier");
Button buttonDelete = new Button("Supprimer");
Button buttonImport = new Button("Importer");
Button buttonExport = new Button("Exporter");

HBox hbox = new HBox(8);
hbox.getChildren().addAll(buttonAdd, buttonModify, buttonDelete, buttonImport,
buttonExport);

borderPane.setBottom(hbox);
```

Un **HBox** range ses éléments horizontalement, sur une seule ligne. On ajoute les boutons dedans avec **getChildren().addAll()**, puis on met le HBox en bas du BorderPane avec **setBottom()**. On a donc un layout (HBox) dans un autre layout (BorderPane), comme dans la diapo 40.

Cours : chapitre 11, diapos 20 (Button), 36 (HBox), 39 et 40 (BorderPane).

Question 10 : espacer et agrandir les boutons

L'espace : on donne la valeur au constructeur, **new HBox(8)** met 8 pixels entre chaque bouton (déjà fait au-dessus).

Prendre toute la largeur : pour chacun des 5 boutons on écrit les 2 lignes données par le TP :

```
import javafx.scene.layout.Priority;

buttonAdd.setMaxWidth(Double.MAX_VALUE);     // autorise l'agrandissement
HBox.setHgrow(buttonAdd, Priority.ALWAYS);   // répartit l'espace dans HBox
```

- Par défaut un bouton a la taille de son texte et ne grandit pas. **setMaxWidth(Double.MAX_VALUE)** enlève cette limite (**Double.MAX_VALUE** est le plus grand double possible).
- **HBox.setHgrow(..., Priority.ALWAYS)** dit au HBox de donner l'espace libre à ce bouton. Comme les 5 boutons ont ALWAYS, l'espace est partagé entre eux.
- **setHgrow** est une méthode **static** : on l'appelle avec le nom de la classe **HBox**, pas avec l'objet hbox.

Sujet du TP, question 10 ; chapitre 3, diapo 30 pour static ; chapitre 7 pour la classe Double.

Question 11 : la fenêtre d'édition openEditor()

On ajoute à **App** une méthode qui ouvre une 2ème fenêtre. Elle sert à deux choses : **créer** un contact (on reçoit **null**) ou **modifier** un contact (on reçoit le contact).

1) La nouvelle fenêtre : une fenêtre = un **Stage**, donc on en crée un nouveau.

```
private void openEditor(Contact toEdit) {
    Stage dlg = new Stage();
```

2) Les Label et les TextField : un **Label** affiche un texte fixe, un **TextField** est une zone où l'utilisateur tape une ligne de texte.

```
Label labelLastName = new Label("Nom :");
TextField textfieldLastName = new TextField();
// pareil pour Prénom, Email et Téléphone
```

3) Pré-remplir si on modifie : si toEdit n'est pas null, on met ses valeurs dans les champs avec **setText()**.

```
if (toEdit != null) {
    textfieldLastName.setText(toEdit.getLastName());
    textfieldFirstName.setText(toEdit.getFirstName());
    textfieldEmail.setText(toEdit.getEmail());
    textfieldPhone.setText(toEdit.getPhone());
}
```

4) La disposition : pour avoir les labels bien alignés à gauche et les champs à droite (comme sur la capture du TP), on utilise un **GridPane**, c'est-à-dire une grille. **add(composant, colonne, ligne)** place le composant dans la case voulue (on compte à partir de 0).

```
GridPane gridPane = new GridPane();
gridPane.add(labelLastName, 0, 0);        // colonne 0, ligne 0
gridPane.add(textfieldLastName, 1, 0);    // colonne 1, ligne 0
gridPane.add(labelFirstName, 0, 1);
gridPane.add(textfieldFirstName, 1, 1);
// ... et ainsi de suite pour les lignes 2 et 3
```

Les boutons OK et Annuler sont dans un HBox, comme demandé. Ensuite on met la grille et le HBox l'un sous l'autre dans un **VBox**. **new VBox(10, gridPane, hboxBoutons)** crée le VBox avec 10 pixels d'espace et y ajoute directement les deux éléments.

```
Button okBtn = new Button("OK");
Button cancelBtn = new Button("Annuler");
HBox hboxBoutons = new HBox(8);
hboxBoutons.getChildren().addAll(okBtn, cancelBtn);

VBox root = new VBox(10, gridPane, hboxBoutons);
Scene scene = new Scene(root, 320, 180);
dlg.setScene(scene);
```

5) Le bouton Annuler : il ferme juste la fenêtre, rien n'est enregistré.

```
cancelBtn.setOnAction(e -> dlg.close());
```

setOnAction() donne le code à exécuter quand on clique. On l'écrit avec une **expression lambda** : **e -> instruction**. **e** est l'événement (le clic), on ne s'en sert pas.

6) Le bouton OK : on récupère le texte des 4 champs avec **getText()**, puis deux cas :

```
okBtn.setOnAction(e -> {
    String ln = textfieldLastName.getText();
    String fn = textfieldFirstName.getText();
    String em = textfieldEmail.getText();
    String ph = textfieldPhone.getText();

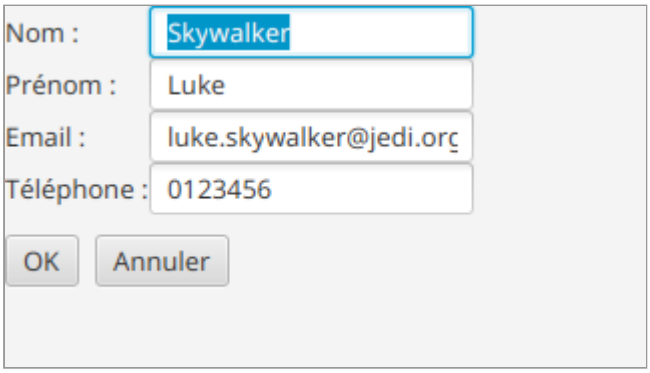
    if (toEdit == null) {
        // Création d'un nouveau contact
        contacts.add(new Contact(ln, fn, em, ph));
    } else {
        // Modification du contact existant
        toEdit.setLastName(ln);
        toEdit.setFirstName(fn);
        toEdit.setEmail(em);
        toEdit.setPhone(ph);
    }
    dlg.close();
});
```

- Il y a plusieurs instructions, donc la lambda a des accolades **{ }**.
- **toEdit == null** : on est en création, on ajoute un nouveau Contact dans **contacts**. Comme c'est une ObservableList, il apparaît tout de suite dans la table.
- Sinon on est en modification : on change les valeurs du contact avec ses setters. Comme ses données sont des StringProperty, la ligne de la table se met à jour toute seule.
- Dans les deux cas on ferme la fenêtre à la fin avec **dlg.close()**.

7) Afficher : on met un titre différent selon le cas, puis **dlg.show()**.

```
if (toEdit == null) {
    dlg.setTitle("Ajouter un contact");
} else {
    dlg.setTitle("Modifier un contact");
}
dlg.show();
```

Cours : chapitre 11, diapos 21 (Label), 22 (TextField), 38 (GridPane), 37 et 61 (VBox), 44 à 47 (setOnAction et lambda), 62 à 64 (ouvrir une nouvelle fenêtre), 68 et 69 (passer des données à une fenêtre : ici on passe toEdit en paramètre).



La fenêtre ouverte avec le bouton Modifier

Question 12 : le bouton Ajouter

```
buttonAdd.setOnAction(e -> openEditor(null));
```

On passe **null** à openEditor(), ce qui veut dire "pas de contact à modifier", donc création.

Cours : chapitre 11, diapo 44 ; chapitre 3 pour null.

Question 13 : le bouton Modifier

```
buttonModify.setOnAction(e -> {
    Contact c = table.getSelectionModel().getSelectedItem();
    openEditor(c);
});
```

table.getSelectionModel().getSelectedItem() renvoie le Contact de la ligne sélectionnée dans la table. On le donne à openEditor() qui ouvre donc la fenêtre en mode modification.

Griser le bouton quand rien n'est sélectionné :

```
buttonModify.disableProperty().bind(
    table.getSelectionModel().selectedItemProperty().isNull()
);
```

C'est un **binding unidirectionnel** : on relie la propriété "désactivé" du bouton à la condition "aucune ligne sélectionnée". Quand la condition est vraie, le bouton est grisé ; dès qu'on clique sur une ligne, elle devient fausse et le bouton redevient actif. On n'a pas besoin d'écrire de if, JavaFX met à jour tout seul.

Grâce à ça, quand on clique sur Modifier, il y a forcément une ligne sélectionnée : c n'est jamais null.

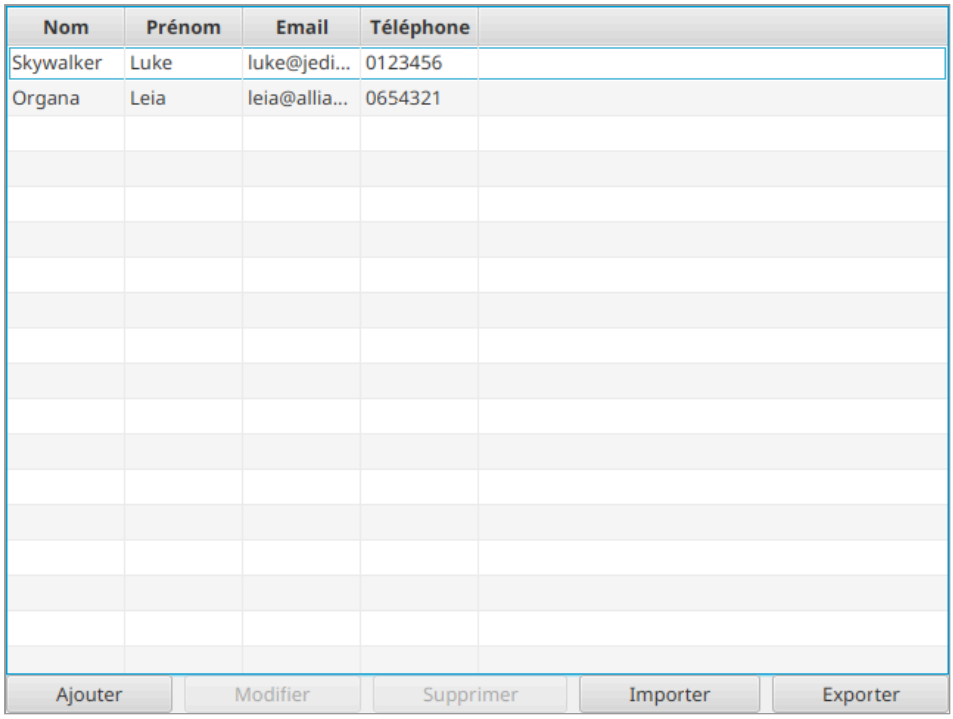
Cours : chapitre 11, diapos 53 et 57 (binding unidirectionnel avec bind).

Question 14 : le bouton Supprimer

```
buttonDelete.setOnAction(e -> {
    Contact c = table.getSelectionModel().getSelectedItem();
    contacts.remove(c);
});
buttonDelete.disableProperty().bind(
    table.getSelectionModel().selectedItemProperty().isNull()
);
```

On récupère le contact sélectionné et on l'enlève de la liste **contacts** avec **remove()**. La table est reliée à la liste, donc la ligne disparaît toute seule. Le bouton est grisé de la même façon que Modifier (question 13).

Cours : chapitre 11, diapo 29 (names.remove) et diapo 57.



La fenêtre principale après un import (aucune ligne sélectionnée : Modifier et Supprimer sont grisés)

Question 15 : le bouton Exporter

```
buttonExport.setOnAction(e -> exporter(stage));
```

Pour que **start()** reste lisible, on écrit le travail dans des méthodes à part. On passe **stage** car le sélecteur de fichier a besoin de savoir à quelle fenêtre il appartient.

1) Le sélecteur de fichier :

```
import javafx.stage.FileChooser;
import java.io.*;

private void exporter(Stage stage) {
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Exporter les contacts");
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("Fichiers
CSV", "*.csv"));

    File f = fileChooser.showSaveDialog(stage);
    if (f != null) {
        ecrireFichier(f);
    }
}
```

- **ExtensionFilter("Fichiers CSV", "*.csv")** : on ne montre que les fichiers .csv.
- Pour exporter on utilise **showSaveDialog()** (fenêtre "Enregistrer sous"), car on veut pouvoir taper le nom d'un nouveau fichier. La diapo 78 montre **showOpenDialog()**, qui sert à ouvrir un fichier existant : on l'utilisera pour l'import.
- Si l'utilisateur clique sur Annuler, la méthode renvoie **null** : dans ce cas on ne fait rien.

2) L'écriture du fichier texte : exactement comme dans le chapitre 9.

```
private void ecrireFichier(File f) {
    BufferedWriter fW;
    try {
        fW = new BufferedWriter(new FileWriter(f));

        // Première ligne : l'en-tête
        fW.write(ENTETE, 0, ENTETE.length());
        fW.newLine();

        // Puis une ligne par contact
        for (int i = 0; i < contacts.size(); i++) {
            Contact c = contacts.get(i);
            String ligne = c.getLastName() + ";" + c.getFirstName() + ";"
                         + c.getEmail() + ";" + c.getPhone();
            fW.write(ligne, 0, ligne.length());
            fW.newLine();
        }
        fW.close();
    } catch (IOException e) {
        afficherErreur("Impossible d'écrire le fichier : " + e.getMessage());
    }
}
```

- **new BufferedWriter(new FileWriter(f))** ouvre le fichier en écriture. S'il existe déjà, son contenu est effacé.
- **write(chaine, 0, chaine.length())** écrit la chaîne, **newLine()** passe à la ligne.
- On parcourt la liste avec **size()** et **get(i)**, comme une ArrayList, et on colle les 4 informations avec des **;** entre elles. Exemple de ligne : **Skywalker;Luke;luke@jedi.org;0123456**
- **close()** ferme le fichier. C'est obligatoire, sinon une partie peut ne pas être écrite.
- Les opérations sur les fichiers peuvent lever une **IOException** (disque plein, pas le droit d'écrire...). Java oblige à la traiter : on met tout dans un **try** et en cas d'erreur on l'affiche à l'utilisateur dans le **catch**.

3) La ligne d'en-tête : on la déclare une seule fois, en haut de la classe App :

```
private static final String ENTETE = "nom;prénom;email;téléphone";
```

final veut dire qu'on ne peut plus la modifier (c'est une constante) et **static** qu'elle appartient à la classe. On s'en sert à l'export ET à l'import : comme c'est la même chaîne, un fichier exporté sera toujours accepté à l'import. On a enlevé les espaces autour des ; pour que l'en-tête ait le même format que les lignes de contacts.

4) Le message d'erreur : on l'écrit dans une méthode car on s'en sert plusieurs fois.

```
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;

private void afficherErreur(String message) {
    Alert alert = new Alert(AlertType.ERROR);
    alert.setTitle("Erreur");
    alert.setHeaderText("Erreur");
    alert.setContentText(message);
    alert.showAndWait();
}
```

Cours : chapitre 11, diapos 77 et 78 (FileChooser), 73 (Alert d'erreur) ; chapitre 9, diapos 8 et 9 (BufferedWriter) ; chapitre 10, diapos 4 à 8 (try / catch, getMessage) ; chapitre 6 (size, get) ; chapitre 3, diapo 30 (static) et chapitre 7 (constante static final).

Question 16 : le bouton Importer

```
buttonImport.setOnAction(e -> importer(stage));
```

1) Le sélecteur de fichier : même chose que pour l'export, mais avec **showOpenDialog()** car on ouvre un fichier qui existe déjà (comme la diapo 78).

```
private void importer(Stage stage) {
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Importer des contacts");
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("Fichiers
CSV", "*.csv"));

    File f = fileChooser.showOpenDialog(stage);
    if (f != null) {
        lireFichier(f);
    }
}
```

2) La lecture du fichier :

```
private void lireFichier(File f) {
    BufferedReader fR;
    try {
        fR = new BufferedReader(new FileReader(f));

        // On vérifie la première ligne
        String ligne = fR.readLine();
        if (ligne == null || !ligne.equals(ENTETE)) {
            fR.close();
            afficherErreur("Ce fichier n'est pas un fichier de contacts.");
            return;
        }

        // On lit les lignes suivantes une par une
        ligne = fR.readLine();
        while (ligne != null) {
            if (ligne.length() > 0) {
                contacts.add(decouperLigne(ligne));
            }
            ligne = fR.readLine();
        }
        fR.close();
    } catch (IOException | StringIndexOutOfBoundsException e) {
        afficherErreur("Erreur de lecture du fichier : " + e.getMessage());
    }
}
```

- **new BufferedReader(new FileReader(f))** ouvre le fichier en lecture, **readLine()** lit une ligne entière. Quand on arrive à la fin du fichier, readLine() renvoie **null**.
- Vérification de l'en-tête : on compare des String avec **equals()** et pas avec == (== compare les adresses des objets, pas le texte). Si le fichier est vide (null) ou si la première ligne n'est pas la bonne, on ferme le fichier, on affiche l'erreur et **return** quitte la méthode, comme demandé.
- La boucle : on lit une ligne, et tant qu'elle n'est pas null on la traite puis on lit la suivante. On saute les lignes vides (longueur 0), par exemple une ligne vide à la fin du fichier.
- Chaque contact créé est ajouté dans **contacts**, donc il apparaît directement dans la table.
- Le **catch** attrape deux exceptions avec le **|** : **IOException** (problème de lecture) et **StringIndexOutOfBoundsException** (une ligne sans assez de ; fait planter substring, voir plus bas). Le programme ne s'arrête pas, il affiche un message.

3) Découper une ligne : on doit passer de **Skywalker;Luke;luke@jedi.org;0123456** à 4 chaînes. On le fait avec **indexOf()** (position du premier ;) et **substring()** (morceau de chaîne entre deux positions), qui sont dans le chapitre 4 :

```
private Contact decouperLigne(String ligne) {
    int p1 = ligne.indexOf(";");
    // ln = ce qui est avant le 1er ; et reste = ce qui est après
    String ln = ligne.substring(0, p1);
    String reste = ligne.substring(p1 + 1, ligne.length());

    int p2 = reste.indexOf(";");
    String fn = reste.substring(0, p2);
    reste = reste.substring(p2 + 1, reste.length());

    int p3 = reste.indexOf(";");
    String em = reste.substring(0, p3);
    String ph = reste.substring(p3 + 1, reste.length());

    return new Contact(ln, fn, em, ph);
}
```

Exemple avec la ligne **Skywalker;Luke;luke@jedi.org;0123456** :

- p1 = 9, donc ln = "Skywalker" et reste = "Luke;luke@jedi.org;0123456"
- p2 = 4, donc fn = "Luke" et reste = "luke@jedi.org;0123456"
- p3 = 13, donc em = "luke@jedi.org" et ph = "0123456"

Si une ligne n'a pas assez de ;, indexOf() renvoie -1 et substring(0, -1) lève une StringIndexOutOfBoundsException : c'est pour ça qu'on l'attrape dans lireFichier().

Limite : si un nom contient lui-même un ;, le fichier sera mal relu. Pour ce TP ce n'est pas demandé, il suffit de ne pas taper de ; dans les champs.

Cours : chapitre 11, diapos 73 et 78 ; chapitre 9, diapos 6 et 7 (BufferedReader) ; chapitre 4, diapos 4 (length), 6 (substring), 8 (indexOf) et 10 (equals) ; chapitre 10, diapo 11 (plusieurs exceptions dans un catch avec |) ; chapitre 2 (while).

Lancer et tester

- Copier **App.java** et **Contact.java** du dossier **TP05/GestionContacts** dans le projet NetBeans (dossier Source Packages, package com.mycompany.gestioncontacts).
- Lancer avec le bouton **Run Project** (triangle vert) ou la touche **F6**.
- Test 1 : Ajouter 2 contacts, ils apparaissent dans la table.
- Test 2 : sans sélection, Modifier et Supprimer sont grisés. Sélectionner une ligne, Modifier, changer le prénom, OK : la ligne change.
- Test 3 : Supprimer une ligne sélectionnée.
- Test 4 : Exporter dans contacts.csv, puis ouvrir ce fichier avec un éditeur de texte pour voir son contenu :

  ```
  nom;prénom;email;téléphone
  Skywalker;Luke;luke@jedi.org;0123456
  Organa;Leia;leia@alliance.org;0654321
  ```

- Test 5 : supprimer les contacts, puis Importer contacts.csv : ils reviennent.
- Test 6 : Importer un fichier .csv dont la première ligne est différente : le message d'erreur s'affiche.

Annexe 1 : Contact.java complet

```
package com.mycompany.gestioncontacts;

import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;

public class Contact {

    // DONNEES MEMBRES : des StringProperty pour qu'elles soient observables
    private StringProperty lastName = new SimpleStringProperty("");
    private StringProperty firstName = new SimpleStringProperty("");
    private StringProperty email = new SimpleStringProperty("");
    private StringProperty phone = new SimpleStringProperty("");

    // Constructeur : on recoit des String et on les range dans les StringProperty
    public Contact(String ln, String fn, String em, String ph) {
        lastName.set(ln);
        firstName.set(fn);
        email.set(em);
        phone.set(ph);
    }

    // Nom
    public String getLastName() { return lastName.get(); }
    public void setLastName(String v) { lastName.set(v); }
    public StringProperty lastNameProperty() { return lastName; }

    // Prenom
    public String getFirstName() { return firstName.get(); }
    public void setFirstName(String v) { firstName.set(v); }
    public StringProperty firstNameProperty() { return firstName; }

    // Email
    public String getEmail() { return email.get(); }
    public void setEmail(String v) { email.set(v); }
    public StringProperty emailProperty() { return email; }

    // Telephone
    public String getPhone() { return phone.get(); }
    public void setPhone(String v) { phone.set(v); }
    public StringProperty phoneProperty() { return phone; }
}
```

Annexe 2 : App.java complet

```
package com.mycompany.gestioncontacts;

import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.VBox;
import javafx.stage.FileChooser;
import javafx.stage.Stage;

import java.io.*;

/**
 * JavaFX App
 */
public class App extends Application {

    // Ligne d'en-tete du fichier CSV (la meme pour l'export et l'import)
    private static final String ENTETE = "nom;prénom;email;téléphone";

    // Liste observable des contacts : toute modification est repercutee dans la
TableView
    private ObservableList<Contact> contacts = FXCollections.observableArrayList();

    @Override
    public void start(Stage stage) {

        // Q3 : titre de la fenetre
        stage.setTitle("Gestion de contacts");

        // Q4 : layout principal
        BorderPane borderPane = new BorderPane();

        // Q8 : la table et ses 4 colonnes
        TableView<Contact> table = new TableView<>();

        TableColumn<Contact, String> lastNameCol = new TableColumn<>("Nom");
        lastNameCol.setCellValueFactory(new PropertyValueFactory<>("lastName"));

        TableColumn<Contact, String> firstNameCol = new TableColumn<>("Prénom");
        firstNameCol.setCellValueFactory(new PropertyValueFactory<>("firstName"));

        TableColumn<Contact, String> mailCol = new TableColumn<>("Email");
        mailCol.setCellValueFactory(new PropertyValueFactory<>("email"));

        TableColumn<Contact, String> phoneCol = new TableColumn<>("Téléphone");
        phoneCol.setCellValueFactory(new PropertyValueFactory<>("phone"));
```

```
        table.getColumns().addAll(lastNameCol, firstNameCol, mailCol, phoneCol);
        table.setItems(contacts);

        borderPane.setCenter(table);

        // Q9 et Q10 : la barre d'action en bas de la fenetre
        Button buttonAdd = new Button("Ajouter");
        Button buttonModify = new Button("Modifier");
        Button buttonDelete = new Button("Supprimer");
        Button buttonImport = new Button("Importer");
        Button buttonExport = new Button("Exporter");

        HBox hbox = new HBox(8);
        hbox.getChildren().addAll(buttonAdd, buttonModify, buttonDelete,
buttonImport, buttonExport);

        buttonAdd.setMaxWidth(Double.MAX_VALUE);
        HBox.setHgrow(buttonAdd, Priority.ALWAYS);
        buttonModify.setMaxWidth(Double.MAX_VALUE);
        HBox.setHgrow(buttonModify, Priority.ALWAYS);
        buttonDelete.setMaxWidth(Double.MAX_VALUE);
        HBox.setHgrow(buttonDelete, Priority.ALWAYS);
        buttonImport.setMaxWidth(Double.MAX_VALUE);
        HBox.setHgrow(buttonImport, Priority.ALWAYS);
        buttonExport.setMaxWidth(Double.MAX_VALUE);
        HBox.setHgrow(buttonExport, Priority.ALWAYS);

        borderPane.setBottom(hbox);

        // Q12 : bouton Ajouter
        buttonAdd.setOnAction(e -> openEditor(null));

        // Q13 : bouton Modifier
        buttonModify.setOnAction(e -> {
            Contact c = table.getSelectionModel().getSelectedItem();
            openEditor(c);
        });
        buttonModify.disableProperty().bind(
            table.getSelectionModel().selectedItemProperty().isNull()
        );

        // Q14 : bouton Supprimer
        buttonDelete.setOnAction(e -> {
            Contact c = table.getSelectionModel().getSelectedItem();
            contacts.remove(c);
        });
        buttonDelete.disableProperty().bind(
            table.getSelectionModel().selectedItemProperty().isNull()
        );

        // Q15 et Q16 : boutons Exporter et Importer
        buttonExport.setOnAction(e -> exporter(stage));
        buttonImport.setOnAction(e -> importer(stage));

        var scene = new Scene(borderPane, 640, 480);
        stage.setScene(scene);
        stage.show();
    }

    // Q11 : fenetre de saisie / modification d'un contact
    // toEdit vaut null pour un nouveau contact
    private void openEditor(Contact toEdit) {
        Stage dlg = new Stage();
```

```
Label labelLastName = new Label("Nom :");
Label labelFirstName = new Label("Prénom :");
Label labelEmail = new Label("Email :");
Label labelPhone = new Label("Téléphone :");

TextField textfieldLastName = new TextField();
TextField textfieldFirstName = new TextField();
TextField textfieldEmail = new TextField();
TextField textfieldPhone = new TextField();

// Si on modifie un contact, on pre-remplit les champs
if (toEdit != null) {
    textfieldLastName.setText(toEdit.getLastName());
    textfieldFirstName.setText(toEdit.getFirstName());
    textfieldEmail.setText(toEdit.getEmail());
    textfieldPhone.setText(toEdit.getPhone());
}

// Les labels dans la colonne 0, les champs dans la colonne 1
GridPane gridPane = new GridPane();
gridPane.add(labelLastName, 0, 0);
gridPane.add(textfieldLastName, 1, 0);
gridPane.add(labelFirstName, 0, 1);
gridPane.add(textfieldFirstName, 1, 1);
gridPane.add(labelEmail, 0, 2);
gridPane.add(textfieldEmail, 1, 2);
gridPane.add(labelPhone, 0, 3);
gridPane.add(textfieldPhone, 1, 3);

Button okBtn = new Button("OK");
Button cancelBtn = new Button("Annuler");
HBox hboxBoutons = new HBox(8);
hboxBoutons.getChildren().addAll(okBtn, cancelBtn);

// Bouton Annuler : on ferme simplement la fenetre
cancelBtn.setOnAction(e -> dlg.close());

// Bouton OK : on recupere les valeurs saisies
okBtn.setOnAction(e -> {
    String ln = textfieldLastName.getText();
    String fn = textfieldFirstName.getText();
    String em = textfieldEmail.getText();
    String ph = textfieldPhone.getText();

    if (toEdit == null) {
        // Creation d'un nouveau contact
        contacts.add(new Contact(ln, fn, em, ph));
    } else {
        // Modification du contact existant
        toEdit.setLastName(ln);
        toEdit.setFirstName(fn);
        toEdit.setEmail(em);
        toEdit.setPhone(ph);
    }
    dlg.close();
});

// La grille au-dessus, les boutons en dessous
VBox root = new VBox(10, gridPane, hboxBoutons);
Scene scene = new Scene(root, 320, 180);
dlg.setScene(scene);

if (toEdit == null) {
    dlg.setTitle("Ajouter un contact");
```

```
        } else {
            dlg.setTitle("Modifier un contact");
        }
        dlg.show();
    }

    // Q15 : export des contacts dans un fichier CSV
    private void exporter(Stage stage) {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Exporter les contacts");
        fileChooser.getExtensionFilters().add(new
FileChooser.ExtensionFilter("Fichiers CSV", "*.csv"));

        File f = fileChooser.showSaveDialog(stage);
        if (f != null) {
            ecrireFichier(f);
        }
    }

    private void ecrireFichier(File f) {
        BufferedWriter fW;
        try {
            fW = new BufferedWriter(new FileWriter(f));

            // Premiere ligne : l'en-tete
            fW.write(ENTETE, 0, ENTETE.length());
            fW.newLine();

            // Puis une ligne par contact
            for (int i = 0; i < contacts.size(); i++) {
                Contact c = contacts.get(i);
                String ligne = c.getLastName() + ";" + c.getFirstName() + ";"
                             + c.getEmail() + ";" + c.getPhone();
                fW.write(ligne, 0, ligne.length());
                fW.newLine();
            }
            fW.close();
        } catch (IOException e) {
            afficherErreur("Impossible d'écrire le fichier : " + e.getMessage());
        }
    }

    // Q16 : import des contacts depuis un fichier CSV
    private void importer(Stage stage) {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Importer des contacts");
        fileChooser.getExtensionFilters().add(new
FileChooser.ExtensionFilter("Fichiers CSV", "*.csv"));

        File f = fileChooser.showOpenDialog(stage);
        if (f != null) {
            lireFichier(f);
        }
    }

    private void lireFichier(File f) {
        BufferedReader fR;
        try {
            fR = new BufferedReader(new FileReader(f));

            // On verifie la premiere ligne
            String ligne = fR.readLine();
            if (ligne == null || !ligne.equals(ENTETE)) {
                fR.close();
```

```
                afficherErreur("Ce fichier n'est pas un fichier de contacts.");
                return;
            }

            // On lit les lignes suivantes une par une
            ligne = fR.readLine();
            while (ligne != null) {
                if (ligne.length() > 0) {
                    contacts.add(decouperLigne(ligne));
                }
                ligne = fR.readLine();
            }
            fR.close();
        } catch (IOException | StringIndexOutOfBoundsException e) {
            afficherErreur("Erreur de lecture du fichier : " + e.getMessage());
        }
    }

    // Transforme "nom;prenom;email;telephone" en objet Contact
    private Contact decouperLigne(String ligne) {
        int p1 = ligne.indexOf(";");
        String ln = ligne.substring(0, p1);
        String reste = ligne.substring(p1 + 1, ligne.length());

        int p2 = reste.indexOf(";");
        String fn = reste.substring(0, p2);
        reste = reste.substring(p2 + 1, reste.length());

        int p3 = reste.indexOf(";");
        String em = reste.substring(0, p3);
        String ph = reste.substring(p3 + 1, reste.length());

        return new Contact(ln, fn, em, ph);
    }

    // Affiche une boite de dialogue d'erreur
    private void afficherErreur(String message) {
        Alert alert = new Alert(AlertType.ERROR);
        alert.setTitle("Erreur");
        alert.setHeaderText("Erreur");
        alert.setContentText(message);
        alert.showAndWait();
    }

    public static void main(String[] args) {
        launch();
    }

}
```